Discussion of Exercise Sheet 4

Exercise 4.4 *One-Looped Sort* **(2 bonus points).** → Peergrading Aufgabe

Consider the following pseudocode whose goal is to sort an array A containing n integers.

Algorithm 1 Input: array $A[0 \dots n-1]$.

```
i ← 0
while i < n do
    if i = 0 or A[i] ≥ A[i − 1] then:
        i ← i + 1
    else
        swap A[i] and A[i − 1]
        i ← i − 1
```

(b) Explain why the algorithm correctly sorts any input array. Formulate a reasonable loop invariant, prove it (e.g., using induction), and then conclude using that invariant that the algorithm correctly sorts the array.

(Master Solution is not that precise ;i)

Hint: *Use the invariant "at the moment when the variable i gets incremented to a new value $i = k$ for the first time, the first k elements of the array are sorted in increasing order".*

Inv(k) = At the moment when the variable i gets incremented to a new value i = k for the first time, the first k elements A[0 .. k-1] are sorted in increasing order

B.C. Let k = 1 At the moment when the variable i gets incremented to i = 1 for the first time, the first element A[0] is sorted in increasing order, because a single element is always trivially sorted. Thus Inv(1) holds

I.H: Assume Inv(k) for some arbitrary but fixed $k \in \{1, .., n-1\}$

I.S: We look at the moment when the variable i gets incremented to i = k for the first time By our I.H. we know that A[0 .. k-1] is sorted

Since i = k < n, we start the next iteration

Case A[i] ≥ A[i-1]. i gets incremented to i = k+1 for the first time

Since A[0 .. k-1] is sorted and A[k] ≥ A[k-1], it follows that the first k+1 elements A[0 .. k] are sorted Thus Inv(k+1) holds.

Case $A[i] < A[i-1]$: The algorithm swaps $A[i]$ repeatedly with its left neighbor $A[i-1]$ as long as its left neighbor is strictly larger, that is, until it is at the correct position This means that the first $k+1$ elements $A[0..k]$ are sorted Then i is incremented until $i = k+1$ for the first time. Thus $Inv(k+1)$ holds

Termination: Assume that $Inv(n)$ holds. At termination when the loop ends, i has the value $i = n$ By $Inv(n)$ the first n elements $A[0..n]$ are sorted

Thus the entire array is sorted

(very annoying and difficult exercise..)

(b) For the following code snippet, compute the number $T(n)$ of calls to f. Give the exact number of calls. You may use summation signs in your expression.

Algorithm 3

```
i ← 0                                   ⇒ but our sum starts at i = 1, as log(0) is undefined
while i ≤ n do                          for i = 0, no calls happen as 1 = k ≤ i = 0
    j ← 1
    while j² ≤ i do                     <=> j ≤ ⌊√i⌋
        j ← j + 1
        k ← 1
        while k ≤ i do                  <=> k ≤ ⌊log i⌋ + 1
            k ← 2k                          k ← k+1
            f()
    i ← i + 1
```

Überlegung: Sei $j = 4$, $i = 10$
$4^2 \not\leq 10$ deshalb soll $4 \not\leq \lfloor\sqrt{10}\rfloor$

Überlegung:

i	calls	log i
1	1	0
2	2	1
3	2	
4	3	2
5	3	
6	3	

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{\lfloor\sqrt{i}\rfloor} \sum_{k=1}^{\lfloor\log i\rfloor+1} 1$$

$$= \sum_{i=1}^{n} \sum_{j=1}^{\lfloor\sqrt{i}\rfloor} \lfloor\log i\rfloor + 1$$

$$= \sum_{i=1}^{n} \lfloor\sqrt{i}\rfloor \cdot (\lfloor\log i\rfloor + 1)$$

(c) Determine a maximally simplified asymptotic bound in Θ notation for the function $T(n)$ from part (b). You may assume that $n \geq 10$.

obere Schranke: $T(n) \leq \sum_{i=1}^{n} \sqrt{n}\ (\log(n)+1) \leq n\sqrt{n}\ (\log(n)+1) \leq n^{1.5}\, 2\log n \leq O(n^{1.5}\log n)$

untere Schranke:

$$T(n) = \sum_{i=1}^{n} \lfloor\sqrt{i}\rfloor \cdot (\lfloor\log i\rfloor + 1)$$

↳ $\lfloor x\rfloor \leq x - 1$

$$\geq \sum_{i=1}^{n} (\sqrt{i} - 1)\ \log(i)$$

↳ start sum only at $i = \lceil \frac{n}{2} \rceil$

$$\geq \sum_{i=\lceil n/2\rceil}^{n} (\sqrt{i} - 1)\ \log(i)$$

↳ lower bound formula from last week

$$\geq \frac{n}{2} \left(\sqrt{\frac{n}{2}} - 1\right) \log\left(\frac{n}{2}\right)$$

↳ $\log(\frac{n}{2}) = \log(n) - \log(2) = \log(n) - 1$

$$\geq \frac{n}{2} \left(\sqrt{\frac{n}{2}} - 1\right) (\log(n) - 1)$$

↳ we are given that $n \geq 10$.
we just pick a constant $= \frac{1}{100}$ small enough so that we can ignore the "−1"

$$\geq \frac{n}{2}\ \frac{1}{100}\sqrt{\frac{n}{2}} \cdot \frac{1}{100}\ \log(n)$$

$$\geq \Omega(n^{1.5}\log n)$$

daraus folgt: $T(n) = \Theta(n^{1.5}\log n)$

Master Theorem

Theorem 1 (Master theorem). *Let $a, C > 0$ and $b \geq 0$ be constants and $T : \mathbb{N} \to \mathbb{R}^+$ a function such that for all even $n \in \mathbb{N}$,*

$$T(n) \leq aT(n/2) + Cn^b.$$

Then for all $n = 2^k$, $k \in \mathbb{N}$, the following statements hold

(i) If $b > \log_2 a$, $T(n) \leq O(n^b)$.

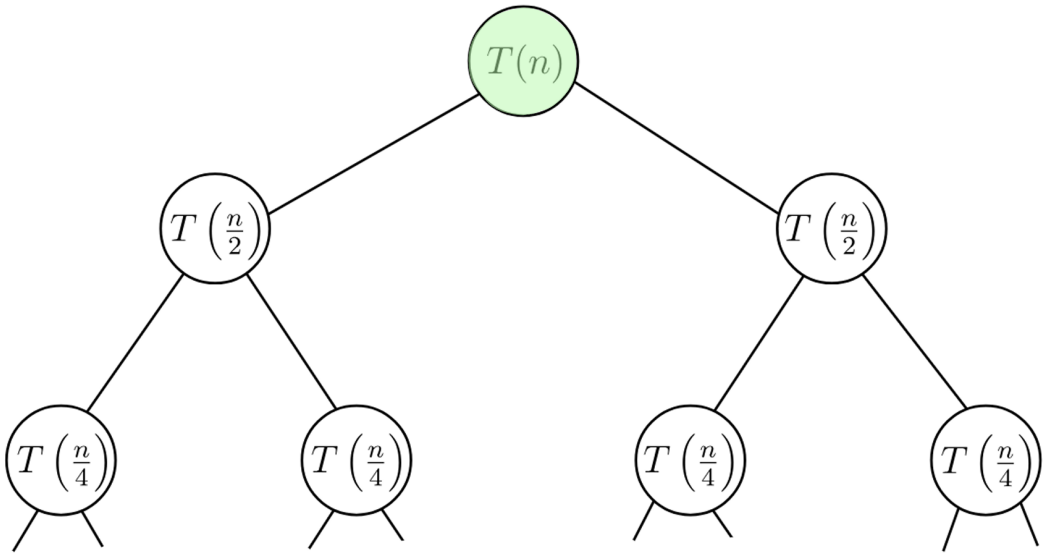
(ii) If $b = \log_2 a$, $T(n) \leq O(n^{\log_2 a} \cdot \log n)$. $\leq O(n^b \log n)$

(iii) If $b < \log_2 a$, $T(n) \leq O(n^{\log_2 a})$.

If the function T is increasing, then the condition $n = 2^k$ can be dropped.

$T(n) \leq \underbrace{a \cdot T\left(\frac{n}{2}\right)}_{\text{Rekursion: Arbeit für Kinder-Knoten}} + \underbrace{C n^b}_{\text{Arbeit für den Knoten selbst}}$

Graph für $a = 2$.



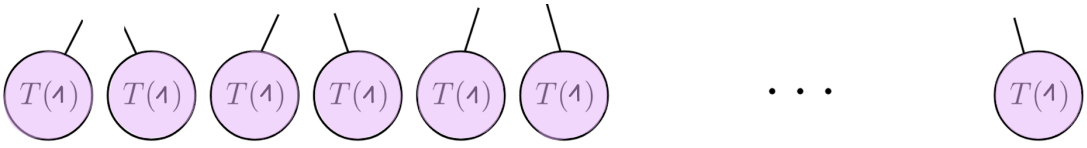
Level 0: $\underbrace{n^b \cdot C}_{\text{Arbeit pro Knoten}}$

Level 1: $\underbrace{a}_{\text{Anzahl Knoten}} \cdot \underbrace{\left(\frac{n}{2}\right)^b \cdot C}_{\text{Arbeit pro Knoten}}$

Level 2: $\underbrace{a^2}_{\text{Anzahl Knoten}} \cdot \underbrace{\left(\frac{n}{2^2}\right)^b C}_{\text{Arbeit pro Knoten}}$

⋮ ⋮ ⋮ ⋮ ⋮ ⋮ ⋮

Level i: $\underbrace{a^i}_{\text{Anzahl Knoten}} \cdot \underbrace{\left(\frac{n}{2^i}\right)^b C}_{\text{Arbeit pro Knoten}}$



Level $\log_2 n$:

$\underbrace{a^{\log_2 n}}_{\text{Anzahl Knoten}} \underbrace{1\, C}_{\text{Arbeit pro Knoten}} = n^{\log_2 a}\, C$

Case $b > \log_2 a$: oberstes Level dominiert, andere Level vernachlässigbar $\Rightarrow O(n^b)$

Case $b = \log_2 a$: alle Level haben genau gleich viel Arbeit $\Rightarrow O(n^b \cdot \underbrace{\log n}_{\text{Anzahl Level}})$

Case $b < \log_2 a$: unterstes Level dominiert, andere Level vernachlässigbar $\Rightarrow O(n^{\log_2 a})$

Beweis:

Potenzgesetz → Distributivgesetz → Potenzgesetz

$$\text{Totale Arbeit: } \sum_{i=0}^{\log_2 n} a^i \left(\frac{n}{2^i}\right)^b = \sum_{i=0}^{\log_2 n} a^i \frac{n^b}{2^{i\,b}} = n^b \sum_{i=0}^{\log_2 n} \frac{a^i}{2^{i\,b}} = n^b \sum_{i=0}^{\log_2 n} \left(\frac{a}{2^b}\right)^i = \ldots$$

Case $b > \log_2 a$: $b > \log_2 a \iff 2^b > a \iff \frac{a}{2^b} < 1$

$$n^b \sum_{i=0}^{\log_2 n} \left(\frac{a}{2^b}\right)^i \leq n^b \sum_{i=0}^{\infty} \left(\frac{a}{2^b}\right)^i = n^b \underbrace{\frac{1}{1-\frac{a}{2^b}}}_{\text{Konstante}} \leq O(n^b)$$

Geometrische Reihe:

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r} \quad \text{falls } r < 1$$

Case $b = \log_2 a$: $b = \log_2 a \iff 2^b = a$

$$n^b \sum_{i=0}^{\log_2 n} \left(\frac{a}{2^b}\right)^i = n^b \sum_{i=0}^{\log_2 n} \underbrace{\left(\frac{a}{a}\right)^i}_{=1} = n^b \sum_{i=0}^{\log_2 n} 1 \leq O(n^b \cdot \log n)$$

($2^b = a$)

Case $b < \log_2 a$: $b < \log_2 a \iff 2^b < a \iff \frac{a}{2^b} > 1$

$$n^b \sum_{i=0}^{\log_2 n} \left(\frac{a}{2^b}\right)^i = n^b \frac{\left(\frac{a}{2^b}\right)^{\log_2(n)+1} - 1}{\left(\frac{a}{2^b}\right) - 1}$$

$$\leq O\left(n^b \cdot \left(\frac{a}{2^b}\right)^{\log_2 n}\right)$$

Geometrische Reihe:

$$\sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1} \quad \text{falls } r > 1$$

↳ Konstanten weglassen

Aufgaben:

Let $T(1) = 3, T(n) = 2T(n/2) + n^2$ for $n > 1$. Using the master theorem, show that

$$T(n) \leq O(n^2).$$

Algorithm 1

```
function A(n)
    i ← 0
    while i < n^2 do
        j ← n
        while j > 0 do
            f()
            f()
            j ← j − 1
        i ← i + 1
    k ← ⌊n/2⌋
    for l = 0 ... 3 do
        if k > 0 then
            A(k)
            A(k)
```

Number of calls =

Algorithm 2

```
function C(n)
    if n ≤ 1 then
        return 0
    j ← 1
    while √j ≤ n^3 do
        f()
        j ← j + 1
    k ← ⌊n/2⌋
    C(k)
    C(k)
    C(k)
```

Number of calls =

Assume that the call $f()$ does not make any recursive calls. Consider the mutually recursive algorithms $g(n)$ and $h(n)$ below, where $n \geq 1$ is a power of 2.

Let $G(n)$ be the number of function calls to f that the function $g(n)$ makes.

Let $H(n)$ be the number of function calls to f that the function $h(n)$ makes.

Give the asymptotic bound in O-Notation for $G(n)$ and for $H(n)$.

Algorithm 2

```
function g(n)
  i ← 1
  while i ≤ n do
    f()
    i ← i + 1
  h(n/2)
  h(n/2)

function h(n)
  if n ≤ 1 then
    return
  i ← 1
  while i − 1 < n^2 do
    f()
    i ← i + 1
  j ← 0
  while j ≤ 6 do
    g(n)
    j ← j + 2
```

(i) $G(n) =$

(ii) $H(n) =$

(i) in (ii) einsetzen:

(ii) in (i) einsetzen:

Sortieren

QUICKSORT$(A[1..n], l, r)$

```
1  if l < r then
2      k ← Aufteilen(A, l, r)        ▷ Teile A[l..r] in zwei Gruppen auf
3      Quicksort(A, l, k − 1)        ▷ Sortiere linke Gruppe
4      Quicksort(A, k + 1, r)        ▷ Sortiere rechte Gruppe
```

AUFTEILEN$(A[1..n], l, r)$

```
1  i ← l                             ▷ Index für die linke Gruppe
2  j ← r − 1                         ▷ Index für die rechte Gruppe
3  p ← A[r]                          ▷ Pivotelement
4  repeat
5      while i < r and A[i] ≤ p do
6          i ← i + 1                 ▷ Suche nächstes Element für rechte Gruppe
7      while j > l and A[j] > p do
8          j ← j − 1                 ▷ Suche nächstes Element für linke Gruppe
9      if i < j then
10         Vertausche A[i] und A[j]  ▷ Vertausche Elemente
11 until i ≥ j                       ▷ Schleife endet, wenn sich i und j „treffen“
12 Vertausche A[i] und A[r]          ▷ Pivotelement an die richtige Stelle tauschen
13 return i                          ▷ Gib Position des Pivotelements zurück
```

Bsp 

Fall $\log a = \log 2 = 1 = b$

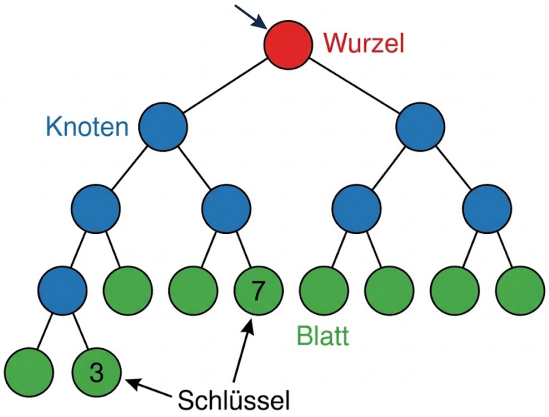
Best Case $T(n) \leq \underbrace{2\,T\left(\frac{n}{2}\right)}_{\text{Rekursion}} + \underbrace{c\,n}_{\text{Aufteilen}}$ ($a=2$, $b=1$) Master Theorem $\Rightarrow$ $O(n \log n)$

Average Case: (zufälliges Pivot) $\underbrace{E[T(n)]}_{\text{Erwartete Laufzeit}} \leq O(n \log n)$

Worst Case: $T(n) \leq \underbrace{T(n-1)}_{\text{Rekursion}} + \underbrace{c\,n}_{\text{Aufteilen}}$ Teleskopieren $\Rightarrow$ $O(n^2)$

Max-Heap

Vollständiger Binärbaum:

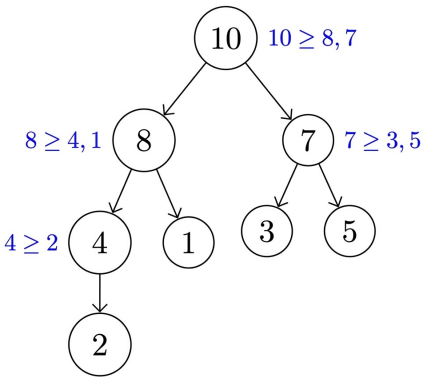


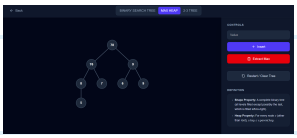
- Alle Knoten, die nicht in den beiden untersten Levels sind, haben genau 2 Kinder.
- Knoten im zweituntersten Level haben 0, 1, oder 2 Kinder
- Knoten im untersten Level haben 0 Kinder
- unterstes Level ist von links nach rechts aufgefüllt

(Wurzel und Blätter sind auch Knoten)

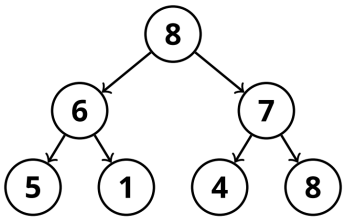
Heap Bedingung:

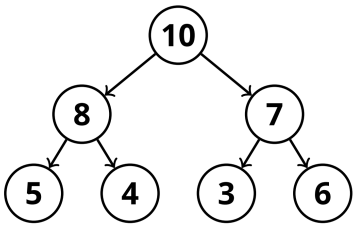
- Schlüssel des Knotens ≥ Schlüssel seiner Kinder

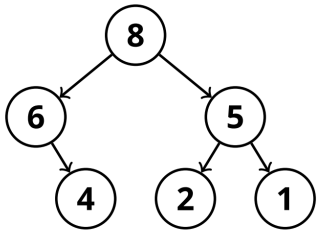


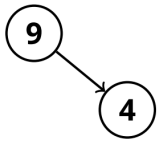


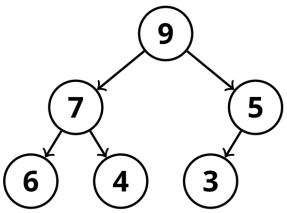
Aufgabe: Entscheide ob folgende Bäume korrekte Max-Heaps sind oder nicht.

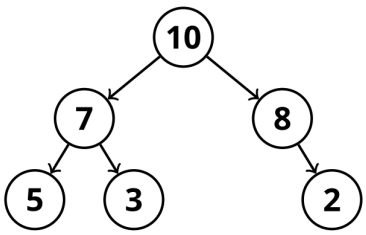


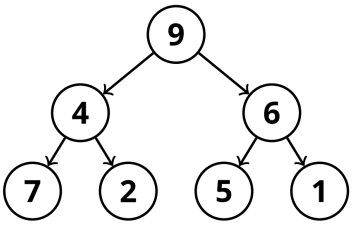


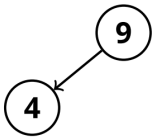












ExtractMax: in $O(\log n)$

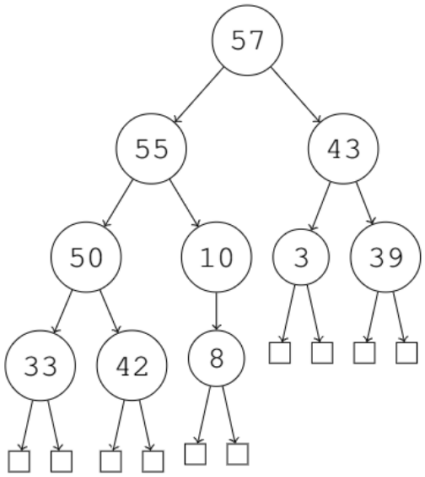
1.) Wurzel (=Max) entfernen

2.) Letztes Blatt an die frei gewordene Stelle der Wurzel tauschen

3.) Die (neue) Wurzel versickern lassen, d.h. so lange mit dem grössten Kind tauschen, bis die Heap-Bedingung wieder gilt.

i) Extract the maximum of the following **max-heap** and do all necessary operations to restore the heap condition (it suffices to only draw the final tree).

2 min an Prüfung

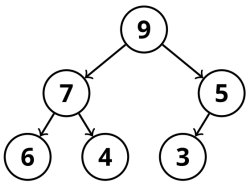


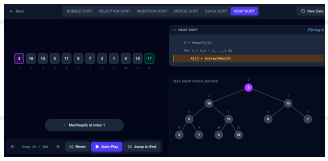
Einfügen: in $O(\log n)$

1.) setze den neuen Knoten an die nächste freie Stelle

2.) tausche ihn mit den Eltern Knoten bis die Heap-Bedingung wieder gilt.

Aufgabe: Füge den Schlüssel 10 ein:





HEAPSORT($A[1..n]$)

1 $H \leftarrow \text{Heapify}(A)$ } O(n)

2 **for** $i \leftarrow n, n-1, \ldots, 1$ **do** } O(n log n)

3 $\quad A[i] \leftarrow \text{ExtractMax}(H)$ O(log n)

▷ *Wandle Array in Heap um.*

▷ *Entferne Elemente aus Heap*

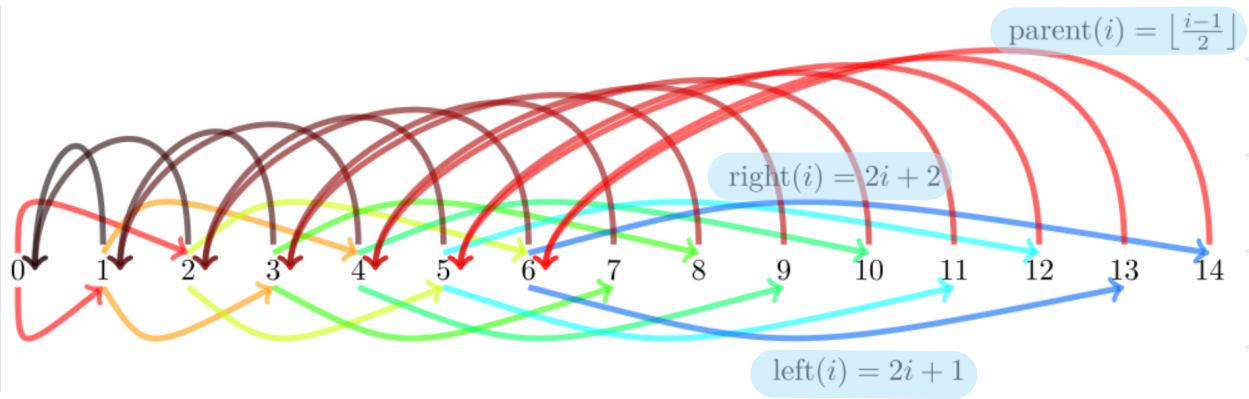
Laufzeit: O(n log n)

Heapify: möglich in O(n)

1.) Fülle die Elemente des Arrays in einen vollständigen Binärbaum

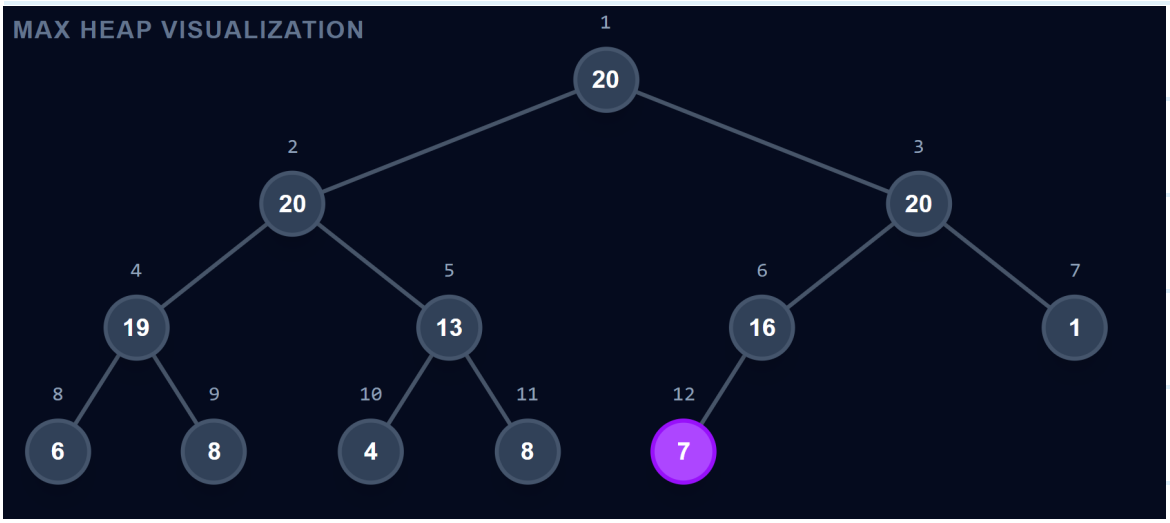
2.) Versickere alle Knoten in aufsteigender Reihenfolge der Levels

Darstellung des Max-Heaps als Array:



Achtung:
In Vorlesung
2i und 2i+1

⇒ In-Place



$A = [a_1 \; a_2 \; a_3 \; .. \; a_n]$

Heapify

tausche Wurzel und letztes Blatt

$[a_n \; a_2 \; a_3 \; .. \; a_1]$

versickern

$[a_2 \; a_n \; a_3 \; .. \; | \; a_1]$

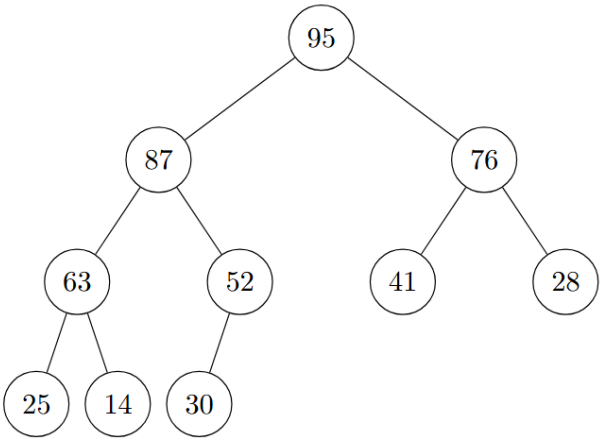
Heap

Inv(i) = grössten i Elemente sind sortiert

Aufgaben:

iii) Draw the **max-heap** obtained from inserting the keys 2, 3, 11, 13, 5, 19 in this order into an empty heap. It suffices to only draw the final heap. If there are several solutions, it is enough to draw only one of them.

Draw the Heap after ExtractMax is called three times



Beispiel für nicht vergleichbasiertes Sortieren:

BUCKET-SORT($A[1..n]$) ▷ *alle Einträge in* $\{1,..,n\}$

funktioniert sonst nicht

```
1 B[1..n] ← [0, 0, ..., 0]
2 for j ← 1, 2, ..., n do
3     B[A[j]] ← B[A[j]] + 1        ▷ Zähle in B[i], wie oft i vorkommt
4 k ← 1
5 for i ← 1, 2, ..., n do
6     A[k, ..., k + B[i] − 1] ← [i, i, .., i]   ▷ Schreibe B[i]-mal den Wert i in A
7     k ← k + B[i]                 ▷ A ist bis Position k+B[i]−1 gefüllt.
```

Laufzeit: $\Theta(n)$

Bsp: $A = [\,3\ \ 4\ \ 4\ \ 6\ \ 5\ \ 1\ \ 2\ \ 6\,]$

$B = [\quad\quad\quad\quad\quad\quad\quad\quad]$

Ausgabe $= [\,1\ \ 2\ \ 3\ \ 4\ \ 4\ \ 5\ \ 6\ \ 6\,]$

Entscheidungsbaum:

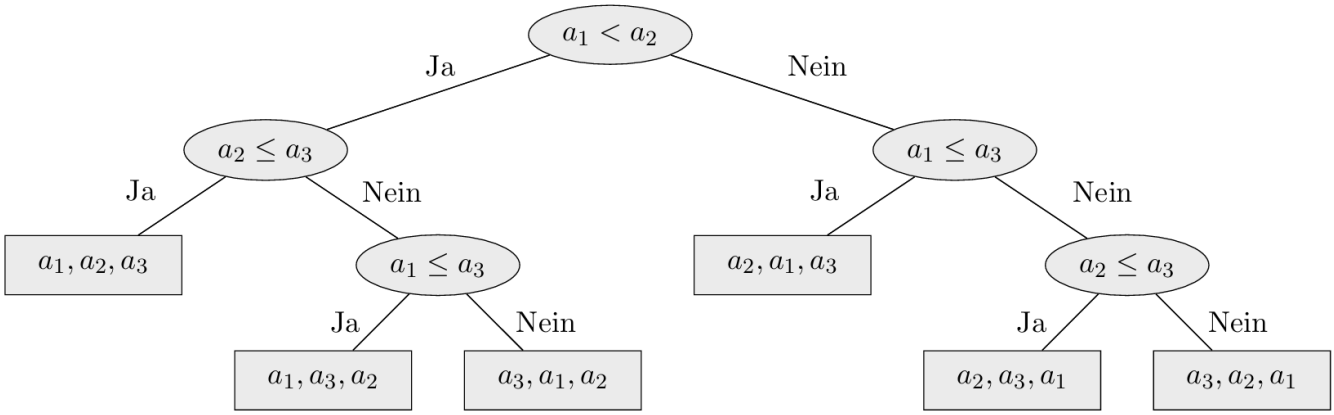


Abb. 2.5 Beispiel Entscheidungsbaum mit Array $[a_1, a_2, a_3]$

Wie viele Blätter (= mögliche Ausgaben = Anzahl Permutationen)? → $n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 1 = n!$

Auf Höhe h gibt es genau 2^h viele Knoten

Wie hoch muss der Baum mindestens sein, damit er $n!$ viele Blätter haben kann?

$$2^h \geq n!$$

↳ $n! \geq \left(\frac{n}{2}\right)^{\frac{n}{2}}$

$$\Rightarrow 2^h \geq \left(\frac{n}{2}\right)^{\frac{n}{2}}$$

↳ auf beiden Seiten log nehmen

$$\Rightarrow h \geq \log\left(\left(\frac{n}{2}\right)^{\frac{n}{2}}\right)$$

↳ log Gesetz

$$\Rightarrow h \geq \frac{n}{2} \cdot \log\left(\frac{n}{2}\right)$$

$$\Rightarrow h \geq \Omega(n \log n)$$

Alternativ:

1 Vergleich $\Leftrightarrow$ wir können genau 2 Permutationen unterscheiden

2 Vergleiche $\Leftrightarrow$ wir können genau 4 Permutationen unterscheiden

i Vergleiche $\Leftrightarrow$ wir können genau 2^i Permutationen unterscheiden

Wir müssen $n!$ Permutationen unterscheiden

$\Rightarrow$ wir brauchen $\log(n!) \geq \Omega(n \log n)$ viele Vergleiche

Abstrackte Datentypen

Abstraktion: bedeutet immer das Vereinfachen/Weglassen bestimmter Details eines Problems

→ Bsp Strassenkarten, Zahlen, Programmiersprachen, Logik, ...

Abstrahter Datentyp: beschreibt <u>was</u> mit den Daten passieren soll, ohne im Detail festzulegen <u>wie</u> genau es passieren soll. (=> Interfaces in EProg)

ADT Liste:

- `insert`(k, L): Fügt den Schlüssel k am Ende der Liste L ein.
- `get`(i, L): Gibt den Speicherort des i-ten Objekts in L zurück.
- `delete`(k, L): Entfernt den Schlüssel k aus der Liste L.
- `insertAfter`(k, k', L): Fügt den Schlüssel k' hinter dem Schlüssel k in die Liste L ein.

Drei Implementierungen des ADT Liste:

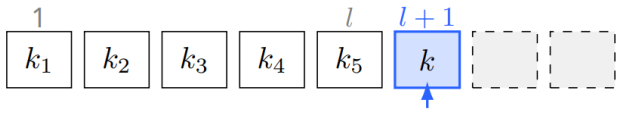
- **Arrays:** Fester, zusammenhängender Speicherbereich mit bekannter Maximalgrösse
- **Einfach verkettete Liste:** ein Element hat immer einen Pointer zum nächsten
- **Zweifach verkettete Liste:** ein Element hat immer einen Pointer zum vorherigen und zum nächsten

Laufzeiten:

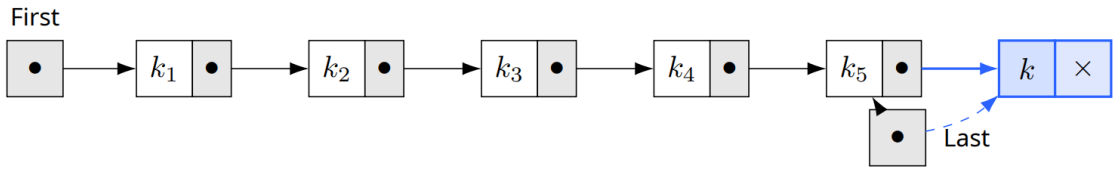
	Array	einf. verlinkte Liste	dopp. verlinkte Liste
`insert`(k, L)	$O(1)$	$O(1)$	$O(1)$
`get`(i, L)	$O(1)$	$O(1) \leq O(\ell)$	$O(1) \leq O(\ell)$
`insertAfter`(k, k', L)	$O(\ell)$	$O(1)$	$O(1)$
`delete`(k, L)	$O(\ell)$	$O(\ell)$	$O(1)$

1. `insert(k, L)` (Am Ende einfügen)

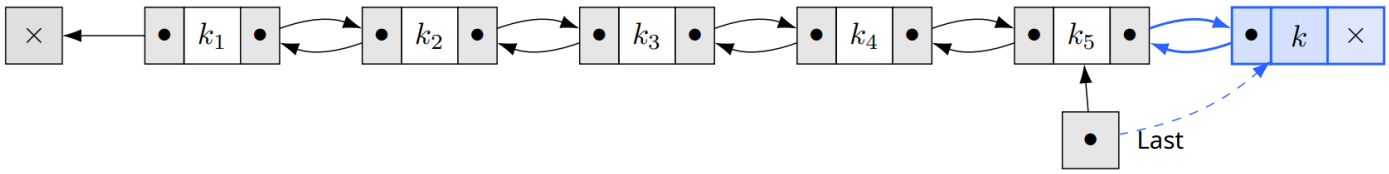
1.1 Array – Laufzeit: $O(1)$



1.2 Einfach verkettete Liste – Laufzeit: $O(1)$

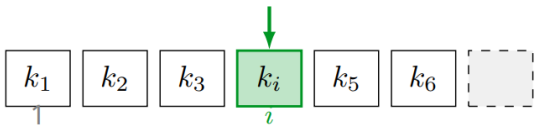


1.3 Doppelt verkettete Liste – Laufzeit: $O(1)$

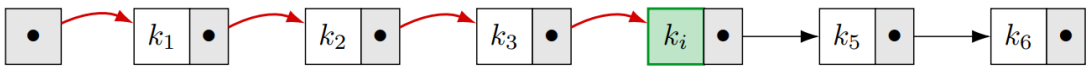


2. `get(i, L)` (Speicherort des i-ten Elements abfragen)

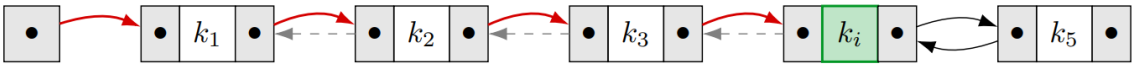
2.1 Array – Laufzeit: $O(1)$



2.2 Einfach verkettete Liste – Laufzeit: $O(l)$

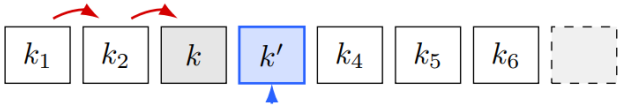


2.3 Doppelt verkettete Liste – Laufzeit: $O(l)$

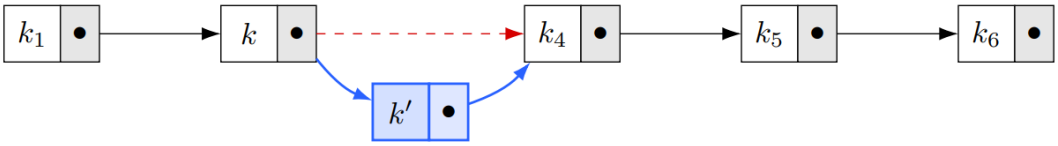


3. `insertAfter(k, k', L)` (Einfügen nach bekanntem Knoten k)

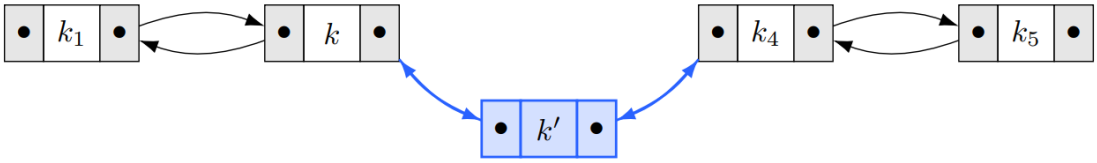
3.1 Array – Laufzeit: $O(l)$



3.2 Einfach verkettete Liste – Laufzeit: $O(1)$

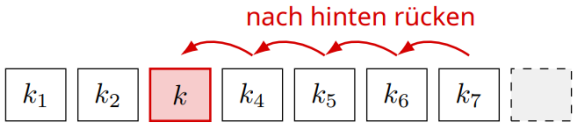


3.3 Doppelt verkettete Liste – Laufzeit: $O(1)$

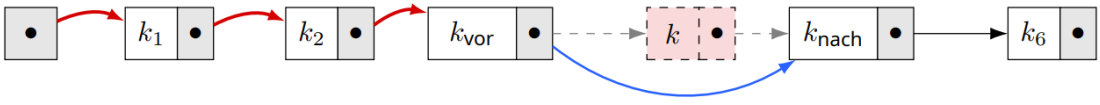


4. `delete(k, L)` (Schlüssel k aus Liste entfernen)

4.1 Array – Laufzeit: $O(l)$



4.2 Einfach verkettete Liste – Laufzeit: $O(l)$



4.3 Doppelt verkettete Liste – Laufzeit: $O(1)$

